

UCS645
PARALLEL & DISTRIBUTED COMPUTING

ASSIGNMENT 4:

Submitted by: Vansh Bhasin (102303270)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Lab 4: Introduction to MPI

UCS645: Parallel & Distributed Computing

TIET Patiala

1. Objective

This lab introduces MPI (Message Passing Interface) programming for distributed memory parallel computing. Four exercises are implemented and tested: Ring Communication, Parallel Array Sum, Finding Maximum and Minimum, and Parallel Dot Product. Each program is compiled using mpicc and run with 4 MPI processes.

2. Environment Setup

MPI was installed on WSL2 (Ubuntu) using the following command:

```
sudo apt-get install mpich
```

Programs were compiled using:

```
mpicc -O3 -o <program> <program>.c
```

And run using:

```
mpirun -np 4 ./<program>
```

3. Exercise 1: Ring Communication

3.1 Problem Statement

Write an MPI program where each process sends a message to the next process in a ring topology. Process 0 starts with value 100, each process adds its rank to the value before passing it on, and the final value returns to Process 0.

3.2 Key Concepts Used

- MPI_Send and MPI_Recv for point-to-point communication
- Modulo operator (rank + 1) % size to wrap around the ring
- Process 0 both initiates and receives the final value

3.3 Output

Process	Action	Value
0	Starting value, sends to Process 1	100
1	Received 100, adds rank 1, sends to Process 2	101
2	Received 101, adds rank 2, sends to Process 3	103
3	Received 103, adds rank 3, sends to Process 0	106
0	Received final value back	106

3.4 Analysis

The value starts at 100 (Process 0 adds rank 0), becomes 101 at Process 1 (adds rank 1), 103 at Process 2 (adds rank 2), 106 at Process 3 (adds rank 3), and returns as 106 to Process 0. Final value = $100 + 0 + 1 + 2 + 3 = 106$, which is correct.

4. Exercise 2: Parallel Array Sum

4.1 Problem Statement

Compute the sum of an array of 100 elements (values 1 to 100) in parallel. Distribute the array using MPI_Scatter, compute local sums, then combine using MPI_Reduce. Expected result is 5050.

4.2 Key Concepts Used

- MPI_Scatter to distribute equal chunks of the array to all processes
- Each process independently computes its local sum
- MPI_Reduce with MPI_SUM to gather the global sum at Process 0

4.3 Output

Process	Elements Handled	Local Sum
0	1–25	325
1	26–50	950
2	51–75	1575
3	76–100	2200
Global	Sum = 5050, Average = 50.50	5050 ✓

4.4 Analysis

The array of 100 elements is divided into 4 equal chunks of 25 elements each. Local sums are 325, 950, 1575, and 2200. Their total is 5050, which matches the expected sum of $1+2+\dots+100 = n(n+1)/2 = 5050$. The average is correctly computed as 50.50.

5. Exercise 3: Finding Maximum and Minimum

5.1 Problem Statement

Each process generates 10 random numbers between 0 and 1000. Find the global maximum and minimum values across all processes, along with which process holds each value, using MPI_MAXLOC and MPI_MINLOC.

5.2 Key Concepts Used

- Each process generates random numbers seeded with time + rank for uniqueness
- MPI_Reduce with MPI_2INT datatype and MPI_MAXLOC/MPI_MINLOC operations
- Struct with value and rank fields to track both the extreme value and its source process

5.3 Output

Process	Generated Numbers	Local Max	Local Min
0	975 929 920 256 816 519 284 445 422 795	975	256
1	888 704 420 814 634 621 496 1000 914 897	1000	420
2	583 500 984 584 133 25 138 152 881 589	984	25
3	625 692 836 377 516 934 673 229 349 875	934	229
Global	Maximum: 1000 (Process 1), Minimum: 25 (Process 2)	1000	25

5.4 Analysis

Process 1 held the global maximum of 1000 and Process 2 held the global minimum of 25. MPI_MAXLOC and MPI_MINLOC correctly identified both the extreme values and their source processes simultaneously, demonstrating the power of MPI reduction operations beyond simple arithmetic.

6. Exercise 4: Parallel Dot Product

6.1 Problem Statement

Compute the dot product of two vectors A = [1,2,3,4,5,6,7,8] and B = [8,7,6,5,4,3,2,1] in parallel. Distribute both vectors using MPI_Scatter, compute partial dot products, then sum using MPI_Reduce. Expected result is 120.

6.2 Key Concepts Used

- MPI_Scatter to distribute equal chunks of both vectors to all processes
- Each process computes its partial dot product independently
- MPI_Reduce with MPI_SUM to sum all partial products at Process 0

6.3 Output

Process	Elements Handled	Partial Dot Product
0	$A[1,2] \cdot B[8,7]$	22
1	$A[3,4] \cdot B[6,5]$	38
2	$A[5,6] \cdot B[4,3]$	38
3	$A[7,8] \cdot B[2,1]$	22
Global	Sum of partial products	120 ✓

6.4 Analysis

Each of the 4 processes handles 2 elements. Partial products are 22, 38, 38, and 22. Their sum is 120, matching the expected result. The computation is verified as:

$$(1 \times 8) + (2 \times 7) + (3 \times 6) + (4 \times 5) + (5 \times 4) + (6 \times 3) + (7 \times 2) + (8 \times 1) = 8 + 14 + 18 + 20 + 20 + 18 + 14 + 8 = 120.$$

7. Conclusion

All four MPI exercises were successfully implemented and verified. The key MPI concepts covered include point-to-point communication (MPI_Send/MPI_Recv), collective communication (MPI_Scatter, MPI_Gather, MPI_Bcast), reduction operations (MPI_Reduce with MPI_SUM, MPI_MAXLOC, MPI_MINLOC), and ring topology communication. All programs produced correct results when run with 4 MPI processes on WSL2 with MPICH.